

Metriq — Senior Codebase Audit

Scope: Full read-only review of `client/` (React/Vite/TS/Tailwind) and `server/` (Express/TS/Prisma/PostgreSQL). **Date:** 2026-09-01 · **Method:** Direct code reading across 34 controllers, 20 routers, 58 Prisma models, 74 client pages, plus dependency and config analysis. Every finding cites file:line and was verified against the current tree (not inferred from names). **Nature of "multi-tenant":** This is a **single-company** internal ops app. There is no company-level tenancy; isolation is by **role + department + row ownership**. "Tenant isolation" below therefore means IDOR / department-scope / role enforcement.

1. Executive Summary

Area	Score	One-line justification
Security	5.5 / 10	Genuinely strong access-control <i>foundation</i> (consistent ownership checks, no IDOR, no SQLi, no committed secrets, solid session revocation) — but undermined by a Critical unauthenticated privilege-escalation (self-signup as an activated Team Lead) and cleartext temp-password storage, plus missing rate-limiting/helmet.
Technical / Architecture	7.5 / 10	Above-average: near-zero <code>any</code> , zod in 23/34 controllers, tested pure KPI math in <code>/lib</code> , code-splitting, clean migrations. Held back by 1000-line "god" controllers, no service layer, and zero authorization/integration tests .
UI / UX	7.5 / 10	Disciplined design system — real shared primitives, semantic tokens, zero raw grays, labeled icon buttons. Weak spots: keyboard a11y (Kanban, modals, drawer), unlabeled <code><select></code> s, and inconsistent loading/error states.

Top 10 highest-priority issues (all areas)

#	Sev	Issue	Where
1	Critical	Unauthenticated <code>POST /signup</code> mints an activated <code>TEAM_LEAD</code> in a real department → full dept PII/revenue/roster access	<code>authController.ts:121-146</code>
2	High	Cleartext temp passwords stored in DB and returned in user-list API responses	<code>adminController.ts:53,334,365,415,488</code>
3	High	No rate limiting / lockout on <code>/login</code> , <code>/forgot-password</code> , <code>/reset-password</code> → online brute-force	<code>routes/auth.ts</code> , <code>app.ts:46-53</code>
4	High	<code>JWT_SECRET</code> falls back to a hardcoded string if env unset → forgeable sessions (verify prod)	<code>lib/auth.ts:5</code>
5	High	Zero authorization / controller / integration / client tests — the RBAC invariant is untested	test suite (see §3.6)
6	High	RBAC enforced by 130 hand-copied inline checks , not <code>requireRole</code> at the router; <code>adminRouter</code> has no role guard	<code>routes/admin.ts:17</code> , controllers
7	High	ITAD revenue/ROI computed from Float money columns (rounding drift in financials)	<code>schema.prisma:1522-1525</code> , <code>lib/financials.ts:104</code>
8	High	<code>AuditLog</code> (fastest-growing table) has no index on its query columns (<code>createdAt</code> , <code>userId</code>)	<code>schema.prisma:646-658</code>
9	Medium	Replayable stateless reset/invite tokens (no single-use)	<code>lib/auth.ts:34-48</code>
10	Medium	Kanban task cards keyboard-inoperable (<code><div onClick></code> , <code>PointerSensor</code> only)	<code>EcommerceBoard.tsx:123,35</code>

Bottom line: The engineering quality is high and the authorization model is well-designed and consistently applied — I found **no IDOR and no injection**. The report's urgency is driven by a *single* but severe misconfiguration (public Team-Lead signup) and a cluster of standard hardening gaps (rate-limiting, helmet, secret fail-open, cleartext temp passwords). Fix items 1–4 this week and the security posture jumps to ~8/10.

2. Security

2.1 CRITICAL — Unauthenticated self-signup creates an activated Team Lead

`server/src/controllers/authController.ts:121-146` (route `routes/auth.ts:14`, public). The signup handler routes `department: 'QA'` to a `PENDING` account awaiting approval (correct), but **any** other value in `['ITAD', 'LEAD_GEN', 'MARKETING', 'CSR', 'TALKLOOP']` creates:

```
role: 'TEAM_LEAD', status: 'ACTIVE', departmentId: dept.id,
memberships: { create: { departmentId: dept.id, role: 'TEAM_LEAD' } }
```

...then signs the user in. **Impact:** any unauthenticated person on the internet picks a real department and instantly becomes its Team Lead — gaining every member's PII (name/email/attendance), the ITAD **bid tracker with awarded revenue**, targets, reports, and **roster management** (invite/remove/reset-password of real employees, which also exposes their cleartext temp password — see 2.2). This is a textbook broken-access-control / privilege-escalation. **Fix:** create department-TL signups as `status: 'PENDING'` and require Super-Admin approval (reuse the QA path), or remove self-serve TL registration entirely and provision via `POST /admin/users`. Add an automated test asserting an unauth signup cannot yield an `ACTIVE` privileged account.

2.2 HIGH — Cleartext temporary passwords stored and returned over the API

Writes: `adminController.ts:334,415,419,488` · **Read back in list responses:** `:53` (`listUsers`), `:264,365` (`listTeamMembers`), `:450` (`inviteTeamMember`) · **Column:** `schema.prisma:143` (`User.tempPassword`, comment: *"visible to the Team Lead until the member changes their password"*). On invite/reset the password is written **both** hashed **and in cleartext**, and the cleartext is returned to any Team Lead/Super Admin listing users. Access is correctly role-restricted (so it's not a cross-role leak), but persisting working credentials in plaintext + shipping them in bulk JSON is a real exposure: a DB dump, a backup, or a compromised TL account yields live passwords, and users reuse passwords across systems. It is nulled only when the member self-changes their password — which many never do. **Fix:** show the generated temp password **once** in the invite/reset HTTP response, store only the hash, and drop the `tempPassword` column. If "must change on next login" UX is needed, use a `mustResetPassword` boolean.

2.3 HIGH — No rate limiting or account lockout on auth endpoints

`server/src/routes/auth.ts` (all of `/login`, `/signup`, `/forgot-password`, `/reset-password` are unauthenticated) and `app.ts:46-53` (no throttle middleware anywhere; `express-rate-limit` is not a dependency). With bcrypt cost 10 and 7-day tokens, an attacker can run unthrottled online password/reset-token guessing and signup spam. **Fix:** add `express-rate-limit` — strict per-IP+email limits on `/login`, `/forgot-password`, `/reset-password`; consider progressive backoff/lockout on repeated failures.

2.4 HIGH (verify) — JWT secret fails open to a hardcoded default

`server/src/lib/auth.ts:5` — `const JWT_SECRET = process.env.JWT_SECRET || 'dev-secret-change-me'`. If `JWT_SECRET` is ever unset in production, **all** session and reset tokens are signed with a publicly-known string, letting anyone forge a Super-Admin token. I could not read the prod `.env` (blocked), so this may be set correctly — but the fail-open default is the risk. **Fix:** fail closed — `if (!process.env.JWT_SECRET) throw new Error('JWT_SECRET required')` in production; keep the dev fallback only when `NODE_ENV !== 'production'`. **Action:** confirm prod sets a strong `JWT_SECRET`.

2.5 MEDIUM — Replayable stateless reset / invite tokens

`server/src/lib/auth.ts:34-48` (`signResetToken` 30 min, `signInviteToken` 7 days; both `kind: 'reset'`), consumed in `authController.ts` `resetPassword`. Reset tokens are self-contained JWTs with no server-side single-use tracking. After a reset, `sessionsValidFrom` kills sessions but the **token itself stays valid** for its remaining TTL and can reset the password again. Anyone who later obtains it (email forward, browser history, Referer — see 2.6) can silently take over the account. **Fix:** make reset single-use — embed a nonce derived from the current `passwordHash` / `sessionsValidFrom` and re-verify at use (one reset invalidates the token), or store a hashed one-time token + expiry and delete on use.

2.6 MEDIUM — No security headers (helmet)

`server/src/app.ts:46-53` — no `helmet`. Missing HSTS, `X-Content-Type-Options: nosniff`, `X-Frame-Options` /CSP (clickjacking), and `Referrer-Policy`. The missing Referrer-Policy compounds 2.5 (reset tokens travel in URLs). **Fix:** `app.use(helmet())` with a suitable CSP; set `Referrer-Policy: no-referrer`.

2.7 MEDIUM — CSV formula (spreadsheet) injection in exports

`server/src/lib/csv.ts:5-16` (`toCsv`), used by `financialsController.ts:67` and a second copy in `attendanceController.ts:626,747`. `toCsv` quotes per RFC-4180 but does not neutralize formula triggers. A user-entered field starting with `= + - @` (a name, tag, brand, deal note) becomes a live formula when the exported CSV is opened in Excel/Sheets, enabling

exfiltration/command execution on the downloader's machine. **Fix:** prefix any cell beginning with `= + - @ \t \r` with a leading apostrophe before quoting.

2.8 MEDIUM — File-upload type policy is a denylist; stored content-type is client-controlled

`server/src/controllers/attachmentsController.ts:14-16` (executable denylist), `:138` (download echoes client-supplied `att.mimeType`). `.svg / .html / .xml` are accepted and the download echoes the attacker-supplied MIME type. **Mitigated today** by `Content-Disposition: attachment ( :139 )`, which forces download rather than inline render — so stored-XSS does not fire currently — but the combination is fragile (any future inline/preview path, or MIME-sniffing without `nosniff` from 2.6, reintroduces it). *Positives:* stored filename is a server-generated UUID (no path traversal), 25 MB cap, and download is ownership-gated. **Fix:** switch to an allowlist of expected types; sniff real content server-side; add `X-Content-Type-Options: nosniff`; force `application/octet-stream` for non-render-safe types.

2.9 MEDIUM — `adminRouter` has no router-level role guard (defense-in-depth)

`server/src/routes/admin.ts:17` — only `adminRouter.use(requireAuth)`. Every handler (including `deleteUser`, `resetUserPassword`, `setUserDepartments`) re-checks role in-body via `requireSuperAdmin` (`adminController.ts:848`), so nothing is exploitable today — but the entire admin surface is one forgotten inline check away from exposure. Contrast `financials.ts:12`, which correctly pins `requireRole('SUPER_ADMIN')` at the router. **Fix:** add baseline `requireRole` guards at the router (SA routes under `requireRole('SUPER_ADMIN')`, TL routes under `requireRole('TEAM_LEAD', 'SUPER_ADMIN')`). See also §3.1.

2.10 MEDIUM — Inconsistent request validation & no pagination bounds

~11 controllers coerce `req.query / req.params` with `as string` and no zod (e.g. `reportsController.ts:44,59,62`, `dashboardController.ts:49-50`, `membersController.ts:46-47`, `analyticsController.ts`, `marketingViewsController.ts`, `tasksController.ts`). No SQLi (Prisma parameterizes), but invalid `start / end` dates flow into `periodRange` → Luxon/Prisma and throw generic 500s, and list endpoints accept no clamped `take / skip`. **Fix:** a shared zod schema for query params (enum `range / department`, ISO-date refinement, clamped pagination).

2.11 MEDIUM/LOW — No request body-size limit

`app.ts:52` — `express.json()` with no `limit` (unbounded JSON bodies → memory-pressure DoS). The attachment path buffers up to 26mb (`routes/attachments.ts`) which is above the handler's own 25 MB check. **Fix:** `express.json({ limit: '1mb' })`; align the raw limit to `MAX_BYTES`; ideally stream uploads to disk.

2.12 LOW — PII written to server logs

`server/src/controllers/attendanceController.ts:340,378,410,439,471` — blocked-gate logs include `${me.name} <${me.email}> resolvedIp=... ua="..."`. Employee email/IP/device in plaintext logs is a data-minimization/retention concern. (No passwords/tokens are logged — those greps were clean.) **Fix:** log user id only; drop or hash email/IP/UA.

2.13 LOW — Login user-enumeration via timing side-channel

`authController.ts login` — `if (!user || !user.passwordHash || !(await verifyPassword(...)))` short-circuits bcrypt for non-existent emails, so a missing account responds measurably faster than a real one despite the uniform 401 message. (`/forgot-password` is correctly constant-response — good.) **Fix:** always run a bcrypt compare against a dummy hash when the user is absent.

2.14 LOW — Dependency vulnerabilities are dev/build-tooling only

`npm audit` reports 16 vulns (3 critical, 7 high), but **every** critical/high maps to build-chain tooling — `vitest`, `vite`, `postcss`, `concurrently`, `shell-quote`, `prisma / @prisma/config`, `brace-expansion`, `nanoid`. **No runtime request-path dependency** (`express`, `jsonwebtoken`, `bcryptjs`, `cors`, `zod`, `luxon`) is flagged. Real production exploitability is low. **Fix:** `npm audit fix` for hygiene; not urgent.

Security — verified GOOD (do not "fix")

- **No IDOR found.** Every read/update/delete-by-id verifies ownership or dept/role scope: attachments (`:123-127`), member profile (`membersController.ts:37-44`), QA evals (`canViewEvaluation`), attendance corrections (double-guarded), bids, todos, feedback, ecommerce tasks/stock, RDP, salaries. Confirmed by tracing every router.
- **No injection.** Exactly one raw query — `routes/health.ts:10 prisma.$queryRaw\ SELECT 1` (parameterized)`. No `$queryRawUnsafe`. `Zero dangerouslySetInnerHTML`` in the client.
- **No mass-assignment.** The only two `data: parsed.data` spreads (`adminController.ts:745`, `rdpController.ts:129`) are constrained by narrow zod schemas — role/departmentId/salary not reachable.
- **No committed secrets.** `git ls-files` shows no `.env / keys/service-account`; `.gitignore` covers them; Google creds loaded from env (`lib/google.ts`).
- **Solid session handling.** `requireAuth` re-loads the account each request and honors `isActive / status / sessionsValidFrom`, so disable/role-change/password-reset revoke live tokens immediately (`middleware/auth.ts:45-60`). Cookie is

`httpOnly` + `sameSite:lax` + `secure` -in-prod. Generic 500s leak no stack/Prisma detail (`app.ts:84-88`).

3. Technical / Architecture / Code Quality

3.1 HIGH — Authorization is imperative, not declarative

`middleware/auth.ts:68` defines a clean `requireRole(...)`, but only **7 of 20 routers use it**; the rest rely on **~130 hand-copied** `me.role !== ... checks` in handlers plus `requireSuperAdmin` (`adminController.ts:848`). This makes authz a copy-pasted invariant — a new endpoint that forgets the check ships an authz hole (exactly the risk in 2.1/2.9). **Fix:** move role gating to `requireRole(...)` at the routers; keep only genuine row-ownership checks in handlers.

3.2 HIGH — No authorization / controller / integration tests; no client tests

13 `*.test.ts` cover only pure functions (`kpi`, `itad`, `leadgen`, `financials`, `auth` (hash+JWT only), `time`, `trends`, `shiftDay`, `attendance*`, `userAgent`, `ip`). There are **0** supertest/API tests (despite `app.ts` being split from `index.ts` specifically to enable them) and **0** client tests. The highest-risk logic — RBAC decisions, multi-step writes, assignment — is entirely uncovered. **Fix (highest-leverage):** supertest API tests hitting each router with member/TL/admin tokens to lock the authz matrix; this also prevents regressions of 2.1.

3.3 HIGH — "God-file" controllers

`attendanceController.ts` (1220 lines/17 handlers), `qaController.ts` (1076/20), `adminController.ts` (1002/**31 handlers across 6+ domains**). High merge-conflict surface, hard to navigate, mixes unrelated concerns. **Fix:** split by domain (`adminUsers`, `adminLeave`, `adminActivity`, ...) — routers are already segmented enough to make this mechanical.

3.4 MEDIUM-HIGH — No process-level crash guards

`server/src/index.ts` starts four cron jobs + `app.listen` with **no** `process.on('unhandledRejection'|'uncaughtException')`. *Positive:* each cron tick is individually `.catch()` -wrapped (`monthlyReportCron.ts:43`, `autoCheckout.ts:107`, `attendanceReminders.ts:144`, `attendanceViolations.ts:203`), so known jobs won't crash the process. But a stray rejection (Prisma pool error, new job bug) takes the process down with only a default trace. **Fix:** add top-level handlers that log and exit cleanly for `uncaughtException` (let the process manager restart).

3.5 MEDIUM — No data-access/service layer

All 34 controllers import `../lib/prisma` and query directly. Business *math* is correctly isolated and tested in `/lib` (intent met), but data-access + authorization live in controllers, which is why RBAC/query logic duplicates and files bloat. **Fix (incremental):** extract per-domain service modules owning Prisma + authz; leave controllers as thin HTTP adapters.

3.6 MEDIUM — Unbounded `findMany` on slow-growth tables

Bounded already (good): `loginEvent` / `auditLog` (`take:500`), `notification` (`take:20`), `leaveDay` (`take:200`). *Unbounded:* `feedback` w/ nested replies (`feedbackController.ts:131`), `blogPost` (`marketingBlogController.ts:41`), `ecommerceTask` (`ecommerceController.ts:208,306`), `salaryRecord` (`financialsController.ts:110`), QA analytics scans (`qaController.ts:626,716,939`); plus `attendanceShift.findMany()` re-issued in 4 handlers (`attendanceController.ts:681,805,935,1017`). **Fix:** add `take` + cursor pagination to list endpoints (date-filtered analytic scans are lower priority).

3.7 MEDIUM — `AuthenticatedRequest.user` optional → 137 `req.user!` assertions

`middleware/auth.ts:12` types `user?: AuthUser`, so every guarded handler writes `req.user!.id` — 137 of 147 server non-null assertions. Safe in practice but defeats the type system exactly at the auth boundary. **Fix:** a typed handler wrapper (or `AuthenticatedRequest` variant) that narrows `user` to non-optional.

3.8 MEDIUM — No client data-fetching layer (185 `useEffect` fetches)

No react-query/SWR; every page hand-rolls loading/error/refetch with no dedup/cache (drives UI findings 4.4/4.5). *Mitigations:* code-splitting is in place (60 `lazy()` pages), `useMemo` / `useCallback` used, and recharts (~384 KB) is confined to 3 chart components — **not** in the eager path. **Fix:** adopt react-query to collapse fetch boilerplate; not urgent.

Technical — verified GOOD

- TS rigor is strong: effectively **zero** `any` / `as any` / `@ts-ignore`; zod validates bodies in 23/34 controllers.
- All 12 `catch {}` blocks are intentional and correct (file-existence 404s, `periodRange` fallback, JWT-verify 401, unique-constraint "already sent") — no swallowed errors.

- Clean migration history: 43 sequential migrations, no `DROP / DELETE / TRUNCATE` , consistent provider.
- `asyncHandler` + centralized error middleware wrap every route.

Three biggest maintainability risks: `adminController.ts` , `attendanceController.ts` , `qaController.ts` (+ `client/.../TeamAttendance.tsx` , 714 lines).

4. Data Model (Prisma)

Datasets are small today (hundreds—thousands of rows); performance items rated by future risk.

4.1 HIGH (future) — `AuditLog` missing indexes on its query columns

`schema.prisma:646-658` has only `@@index([entityType, entityId])` , but the activity viewer (`adminController.ts:898-903`) filters `createdAt` (range) + optional `userId / entityType` , `orderBy createdAt desc` . `AuditLog` is append-only and will become the largest table → full scan + sort. `LoginEvent` (:673-674) is correctly indexed — mirror it. **Fix:** add `@@index([createdAt])` and `@@index([userId, createdAt])` .

4.2 HIGH (correctness) — Money stored as `Float` on `Bid`

`schema.prisma:1522-1525` — `priceQuoted` , `awardedPrice` , `bidBondAmount` are `Float` . `awardedPrice` feeds ITAD **revenue/ROI** in Financial Reports (`lib/financials.ts:104` , `reduce(... + (b.awardedPrice ?? 0))`). `Float` accumulation causes rounding drift in reported money. (Newer columns are correct: `SalaryRecord.monthlyCost` and `AdCampaign.spend` are `Decimal(12,2)` — `Bid` is the inconsistent one.) **Fix:** migrate the three `Bid` columns to `Decimal @db.Decimal(12,2)` .

4.3 MEDIUM — Multi-write attendance reconciliation not transactional

`attendanceController.ts:264-277` (`reconcileStaleCheckIn`): three sequential writes (merge clash day, `breakEntry.updateMany` , vacate stale day) with no `$transaction` . A mid-sequence failure corrupts attendance history (which drives late/break/payroll-adjacent logic). **Fix:** wrap in `prisma.$transaction(...)` .

4.4 MEDIUM — Breakdown "replace" is delete-then-recreate outside a transaction

Same pattern in four controllers — `leadgenController.ts:161-177` , `talkloopController.ts:141-154` , `marketingActivityController.ts:142-151` , `ecommerceController.ts:144-158` — `upsert` → `deleteMany` → `createMany` as separate awaits. A failed `createMany` after `deleteMany` silently wipes the user's per-vertical/country/platform/line breakdown while the parent survives. **Fix:** wrap `upsert` + `deleteMany` + `createMany` in `$transaction` .

4.5 MEDIUM (future) — N+1: `submittedToday` per member in a loop

`notificationsController.ts:59,137-138` iterate members and `await submittedToday(...)` each (a `findUnique` per member, :10-19). `getNotSubmitted` runs on every TL/SA dashboard/bell poll, and Super Admin loads *all* members → O(members) queries per poll. **Fix:** batch one `findMany({ where: { userId: { in: ids }, date: today } })` per table, then in-memory set check.

4.6 MEDIUM — `LoginEvent` cascade-deletes the sign-in audit trail

`schema.prisma:664` `onDelete: Cascade` — deleting a user erases their login history, defeating the audit feature. `AuditLog.user` was correctly made optional + `SetNull` (:648-649); `LoginEvent` is inconsistent. (Mitigated because users are soft-deleted via `isActive` , not hard-deleted → MEDIUM.) **Fix:** make `LoginEvent.userId` optional + `onDelete: SetNull` .

4.7 MEDIUM (unsure) — Mixed `onDelete` policy blocks/harms hard user deletion

Required user relations with default `Restrict` — `QaEvaluation.evaluator/agent` (:1245,1247), `Bid.agent` (:1515), `Feedback.author/recipient` (:1144,1146) — make `user.delete()` throw, while other relations `Cascade` (:275). So a hard delete both fails *and* would wipe daily/attendance history — worst of both. Likely intentional (soft-delete only), but the policy is inconsistent. **Fix:** commit to soft-delete as the contract (ensure no `user.delete()` path exists) or standardize `onDelete` across user relations.

4.8 LOW (future) — Missing narrow indexes

`StockRequest.assignedToId` filtered on every Ecommerce bell poll (`notificationsController.ts:101`) but only `@@index([departmentId, status])` exists (:1445). QA unread/coaching filters (:78,85) have usable index prefixes but no exact composite. **Fix:** `@@index([assignedToId, status])` ; `@@index([agentId, status])` on QA — only once volume grows.

Data model — verified GOOD

- **One-entry-per-user-per-day is airtight:** `@@unique([userId,date])` on every daily-entry model (ITAD :293 , `LeadGen` :319 , `Talkloop` :355 , `SEO` :751 , `Content` :768 , `Social` :792 , `Ecommerce` :1370) + `AttendanceDay` / `LeaveDay` / `Holiday` .

- Daily-entry models well-indexed; breakdown children correctly Cascade from their parent; QA snapshot rows Cascade from evaluation.
- `setUserDepartments` (`adminController.ts:220-234`) and QA eval creation (`qaController.ts:349-380`) are correctly atomic.
- Idempotency keys prevent double-sends (`SentMonthlyReport @unique([month,department])`), attendance reminder/violation dedupe keys).

5. UI / UX

5.1 HIGH (a11y) — Kanban cards are not keyboard-operable

`EcommerceBoard.tsx:123` (and `MarketingBoard.tsx`) render the card as `<div onClick={() => onOpen(task.id)}>` with no `role="button"` , `tabIndex` , or key handler — the board's primary action is mouse-only. Both boards register only `PointerSensor` (`EcommerceBoard.tsx:35` , `MarketingBoard.tsx:60`) so drag-to-move is keyboard-inaccessible. (Mitigation: each card has a keyboard-reachable status `<select>` .) **Fix:** make the card a real button (or add `role + tabIndex +Enter/Space`); add `KeyboardSensor` with `sortableKeyboardCoordinates` .

5.2 MEDIUM (a11y) — Native `<select>` /inputs lack programmatic labels

66 `<select>` across pages with `htmlFor` count = 0 (e.g. `TeamMembers.tsx:382-383` , `AdminUsers.tsx:127` , `TeamAttendance.tsx:410,569` , all ecommerce/rdp filters); the `<textarea>` in `ItadDailyLog.tsx:170` (label :169 unassociated). Screen readers announce these unlabeled. **Fix:** add `id + htmlFor / aria-label` ; ideally a shared `Select` that auto-associates like `TextField` does.

5.3 MEDIUM (design-system) — No shared `Select` ; copy-pasted class with drift

A real `Button` / `TextField` exist, but no `Select` . The field class is redefined as a local `const sel / inputCls` in 10+ files (`TeamMembers.tsx:42` , `AdminActivity.tsx:12` , `EcommerceBoard.tsx:153,220` , `RdpRecords.tsx:18` , `TeamAttendance.tsx:705` ...), and the copies have **drifted** (`ring-4 ring-primary/10` vs `ring-2 ring-primary/20`), so focus rings differ across screens. **Fix:** extract `Select` + a shared `fieldClass` into `components/ui` .

5.4 MEDIUM (polish) — No Skeleton; bare "Loading..." text

The plan specifies a `loading_skeleton` , but none exists; ~60 files render plain `Loading...` (e.g. `ItadDailyLog.tsx:123`), causing layout jump. **Fix:** build a `Skeleton` primitive and use it on `DataTable`-backed pages.

5.5 MEDIUM (UX) — Fetch error states largely absent

Only 2 pages track an error variable; most data pages toast-and-continue or silently render empty on failure (e.g. `ItadDailyLog.tsx:86` shows the form zeroed after a load error, indistinguishable from an empty day; no retry). **Fix:** a standard error state + retry affordance, applied consistently.

5.6 MEDIUM (a11y) — Modal has no focus trap / focus restore

`components/ui/Modal.tsx` sets `role="dialog"` , `aria-modal` , Escape, labeled close (good) but never moves focus into the dialog, doesn't trap Tab, and doesn't restore focus on close — keyboard/SR users can Tab behind the overlay. **Fix:** focus first element on open, cycle Tab within, restore on close.

5.7 MEDIUM (a11y) — Mobile nav drawer has no focus trap / `aria-modal` / Escape

`AppShell.tsx:82-99` — the off-canvas drawer closes only on backdrop click or route change; no Escape, no `role="dialog"` / `aria-modal` , no focus containment. (The `ClockWidget` dropdown, `ClockWidget.tsx:74-85` , does this right — reuse that discipline.) **Fix:** add Escape, `aria-modal` , and focus trap to the drawer.

5.8 LOW — Misc

- `DataTable.tsx:115` headers lack `scope="col"` (:151-155 use `<td colSpan>` for section headers). Otherwise the table is solid (sticky header, `overflow-x-auto` , `tabular-nums`).
- `AppShell.tsx:95` — collapsing the desktop sidebar hides nav entirely (`lg:hidden`) rather than a 64px icon rail; discoverability nick.
- `SocialPlanner.tsx:190,195` / `EditorialCalendar.tsx:65,70` — `grid-cols-7` with no responsive/overflow handling → ~45px day cells on a phone.
- No dedicated `mobile_daily_entry` page (spec lists one); daily logs do reflow responsively, so functional but not the tailored screen implied.
- `Landing.tsx:332` — footer renders `<a href="#">` placeholder links.

UI/UX — verified GOOD

Excellent token discipline (0 raw grays, semantic tokens throughout, `tabular-nums` on metrics); real `Button` (variants/sizes/focus-visible) and `TextField` / `PasswordField` (auto `htmlFor` via `useId` , `aria-invalid` , inline errors, labeled show/hide); **zero** unlabeled icon-only buttons; wide tables consistently wrapped in `overflow-x-auto` ; forms handle submit state + draft persistence well; `ink-muted` on white $\approx 4.6:1$ (AA pass).

6. Prioritized Remediation Roadmap

Pri	Sev	Area	Finding	File(s)	Effort
1	Critical	Sec	Public signup → activated Team Lead (§2.1)	<code>authController.ts:121-146</code>	S
2	High	Sec	Cleartext temp passwords stored + returned (§2.2)	<code>adminController.ts:53,334,415,488</code> ; <code>schema.prisma:143</code>	M
3	High	Sec	No auth rate-limiting/lockout (§2.3)	<code>routes/auth.ts</code> , <code>app.ts</code>	S
4	High	Sec	JWT secret fail-open default (§2.4)	<code>lib/auth.ts:5</code>	S
5	High	Tech	No authz/integration/client tests (§3.2)	test suite	L
6	High	Tech/Sec	RBAC inline, not at router; <code>adminRouter</code> no role guard (§3.1, §2.9)	<code>routes/admin.ts:17</code> , controllers	M
7	High	Data	Bid money as Float → revenue drift (§4.2)	<code>schema.prisma:1522-1525</code>	S
8	High	Data	AuditLog missing indexes (§4.1)	<code>schema.prisma:646-658</code>	S
9	High	UI	Kanban cards keyboard-inoperable (§5.1)	<code>EcommerceBoard.tsx:123,35</code>	S
10	Med	Sec	Replayable reset/invite tokens (§2.5)	<code>lib/auth.ts:34-48</code>	M
11	Med	Sec	No helmet / security headers (§2.6)	<code>app.ts</code>	S
12	Med	Sec	CSV formula injection (§2.7)	<code>lib/csv.ts:5-16</code>	S
13	Med	Sec	Upload denylist + client MIME echo (§2.8)	<code>attachmentsController.ts:14,138</code>	M
14	Med	Sec	Query validation + pagination bounds (§2.10)	~11 controllers	M
15	Med	Sec	No JSON body-size limit (§2.11)	<code>app.ts:52</code>	S
16	Med-High	Tech	No process crash guards (§3.4)	<code>index.ts</code>	S
17	Med	Data	Non-transactional multi-writes (§4.3, §4.4)	attendance + 4 breakdown controllers	M
18	Med	Data	N+1 <code>submittedToday</code> in loop (§4.5)	<code>notificationsController.ts:59,137</code>	S
19	Med	Data	LoginEvent cascade wipes audit trail (§4.6)	<code>schema.prisma:664</code>	S
20	Med	Tech	God-file controllers (§3.3)	admin/attendance/qa controllers	L
21	Med	UI	Unlabeled selects/inputs (§5.2)	many pages	M
22	Med	UI	No shared Select; drifted styles (§5.3)	10+ files	M
23	Med	UI	No Skeleton; inconsistent loading (§5.4)	~60 files	M
24	Med	UI	Missing/absent error states (§5.5)	data pages	M
25	Med	UI	Modal + drawer focus trap (§5.6, §5.7)	<code>Modal.tsx</code> , <code>AppShell.tsx</code>	M
26	Med	Tech	Unbounded list <code>findMany</code> (§3.6)	see §3.6	M
27	Low	Sec	PII in logs (§2.12); login timing (§2.13); npm audit hygiene (§2.14)	attendance/auth controllers	S
28	Low	Data	Narrow indexes; onDelete policy (§4.7, §4.8)	schema	S
29	Low	UI	DataTable <code>scope</code> , calendar responsive, dead footer links (§5.8)	see §5.8	S

Pri	Sev	Area	Finding	File(s)	Effort
30	Low	Tech	<code>req.user!</code> typing; helper duplication; client fetch layer (§3.5, §3.7, §3.8)	server + client	M

Effort: $S \approx <1/2 \text{ day}$ · $M \approx 1/2\text{--}2 \text{ days}$ · $L \approx \text{multi-day}$.

Suggested sequencing: Items **1–4** are same-day/one-sprint security fixes with outsized risk reduction — do them first. **5–6** (test the authz matrix, then push RBAC to the routers) make every later change safe and close the drift risk behind item 1. **7–8** protect financial correctness and the audit table before data grows. The remainder is steady hardening/quality work.

Report generated from a read-only review. No code was modified. Items marked "unsure" (§2.4 prod secret, §4.7 delete policy, §5.8 mobile spec) need a maintainer decision or a prod-config check to resolve.